📈 Model 2 — Demand Forecasting

ARIMA + SARIMA + Prophet (3 Time Series Models)

Input: `data_ts_weekly.csv` · **Output:** `data_forecast.csv` + `models/`

> We run all 3 models, compare RMSE, best model wins → final forecast

Cell 1 — Install & Import

```
# Prophet needs separate install
!pip install prophet --quiet
!pip install pmdarima --quiet   # auto_arima for best ARIMA params

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from sklearn.metrics import mean_squared_error, mean_absolute_error

C = ["#534AB7","#1D9E75","#D85A30","#BA7517","#185FA5"]
plt.rcParams.update({
    "figure.figsize"    : (14, 5),
    "axes.spines.top"   : False,
    "axes.spines.right" : False,
    "font.size"         : 11,
})
print("✓ Libraries ready")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 689.1/689.1 kB 30.1 MB/s eta 0:00:00
✓ Libraries ready
```

Cell 2 — Load Weekly Time Series

```
from google.colab import drive
drive.mount('/content/drive')

SAVE = "/content/drive/MyDrive/retail_project/"
```

```
ts = pd.read_csv(SAVE + "data_ts_weekly.csv", parse_dates=["ds"])
ts = ts.sort_values("ds").reset_index(drop=True)

print(f"Shape          : {ts.shape}")
print(f"Date range     : {ts['ds'].min().date()} → {ts['ds'].max().date()}")
print(f"Total weeks    : {len(ts)}")
print(f"Avg revenue/wk : £{ts['y'].mean():,.0f}")
print(f"Max revenue/wk : £{ts['y'].max():,.0f}  ← Christmas spike")
print(f"Min revenue/wk : £{ts['y'].min():,.0f}")
ts.head(5)

# From EDA we know:
# 106 weeks of data | strong Christmas spike Nov-Dec every year
# This seasonality is WHY we need SARIMA and Prophet, not just ARIMA
```

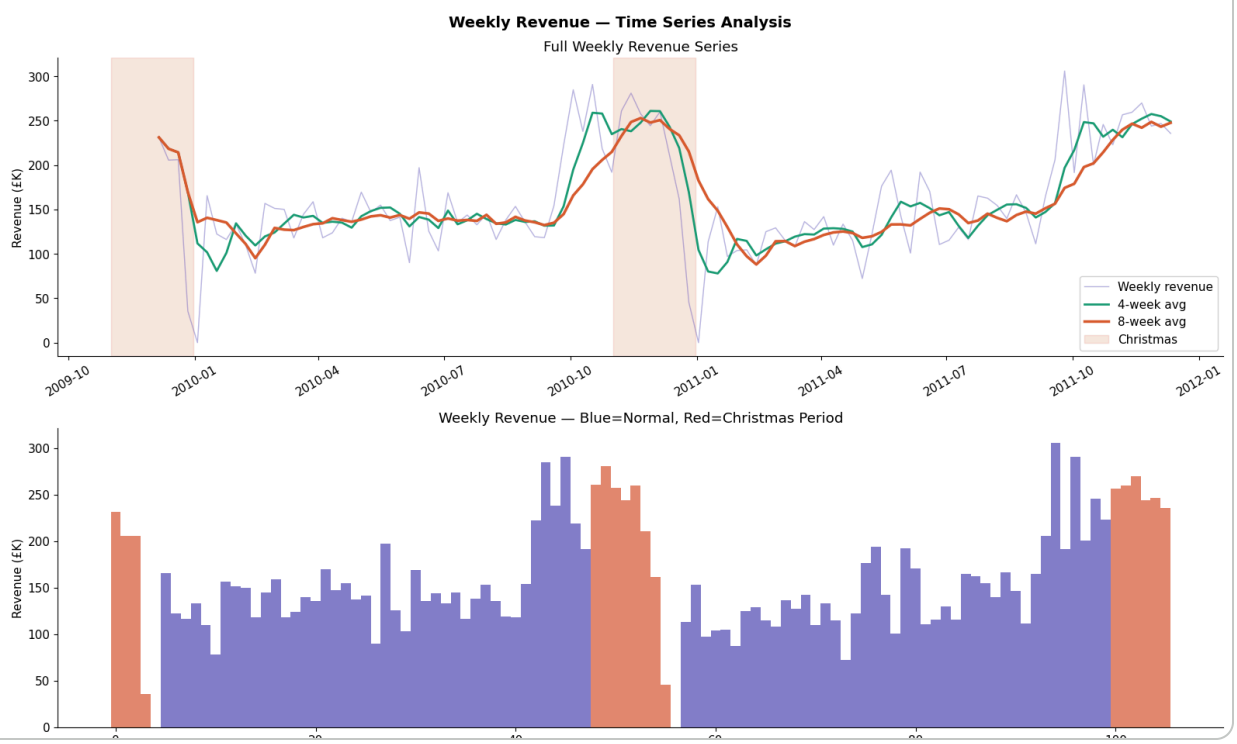
```
Mounted at /content/drive
Shape          : (106, 6)
Date range     : 2009-12-06 → 2011-12-11
Total weeks    : 106
Avg revenue/wk : £158,683
Max revenue/wk : £305,842  ← Christmas spike
Min revenue/wk : £0
```

	ds	y	rolling_4w	rolling_8w	month	is_christmas
0	2009-12-06	231137.45	231137.450000	231137.450000	12	1
1	2009-12-13	205533.06	218335.255000	218335.255000	12	1
2	2009-12-20	206008.22	214226.243333	214226.243333	12	1
3	2009-12-27	35350.84	169507.392500	169507.392500	12	1
4	2010-01-03	0.00	111723.030000	135605.914000	1	0

Cell 3 — Visualize the Time Series

```
fig, axes = plt.subplots(2, 1, figsize=(16, 10))
fig.suptitle("Weekly Revenue — Time Series Analysis", fontsize=14, fontweight="

# Top: full series with moving averages
axes[0].plot(ts["ds"], ts["y"]/1000, alpha=0.4, color=C[0], lw=1, label="Weekly
axes[0].plot(ts["ds"], ts["rolling_4w"]/1000, color=C[1], lw=2, label="4-week a
axes[0].plot(ts["ds"], ts["rolling_8w"]/1000, color=C[2], lw=2.5, label="8-week

# Shade Christmas periods
```

```
for yr in [2009, 2010]:
    axes[0].axvspan(pd.Timestamp(f"{yr}-11-01"), pd.Timestamp(f"{yr}-12-31"),
                    alpha=0.15, color=C[2],
                    label="Christmas" if yr==2009 else "")

axes[0].set_title("Full Weekly Revenue Series")
axes[0].set_ylabel("Revenue (£K)")
axes[0].legend()
axes[0].tick_params(axis="x", rotation=30)

# Bottom: Christmas weeks highlighted
ts["is_christmas"] = ts["ds"].dt.month.isin([11, 12]).astype(int)
colors_bar = [C[2] if x else C[0] for x in ts["is_christmas"]]
axes[1].bar(range(len(ts)), ts["y"]/1000, color=colors_bar, alpha=0.7, width=1)
axes[1].set_title("Weekly Revenue — Blue=Normal, Red=Christmas Period")
axes[1].set_ylabel("Revenue (£K)")
axes[1].set_xlabel("Week number")

plt.tight_layout()
plt.show()

# OBSERVATION:
# Clear Christmas spike every Nov-Dec → annual seasonality (period = 52 weeks)
# Revenue grows from 2009 to 2011 → upward trend
# Both trend + seasonality present → SARIMA(m=52) and Prophet will handle this
```

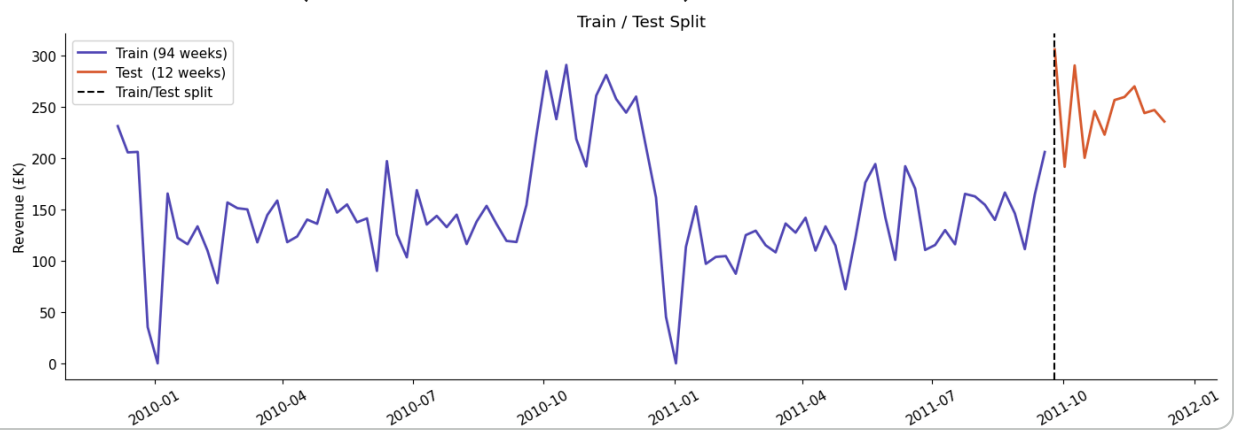


Cell 4 — Train / Test Split

> Last 12 weeks = test set. Train on everything before.

```
TEST_WEEKS = 12   # hold out last 12 weeks for evaluation

train = ts.iloc[:-TEST_WEEKS].copy()
test  = ts.iloc[-TEST_WEEKS:].copy()

print(f"Train : {len(train)} weeks  ({train['ds'].min().date()} → {train['ds'].
print(f"Test  : {len(test)}  weeks  ({test['ds'].min().date()}  → {test['ds'].m

# Visualize split
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(train["ds"], train["y"]/1000, color=C[0], lw=2, label=f"Train ({len(tra
ax.plot(test["ds"],  test["y"]/1000,  color=C[2], lw=2, label=f"Test  ({len(tes
ax.axvline(test["ds"].iloc[0], color="black", lw=1.5, linestyle="--", label="Tr
ax.set_title("Train / Test Split")
ax.set_ylabel("Revenue (£K)")
ax.legend()
ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()

# OBSERVATION:
# Test set covers late 2011 — includes part of Christmas peak
# A good model should predict this spike correctly
```

```
Train : 94 weeks  (2009-12-06 → 2011-09-18)
Test  : 12  weeks  (2011-09-25  → 2011-12-11)
```



Cell 5 — Helper: Evaluation Metrics

```
def evaluate(actual, predicted, model_name):
    """Calculate RMSE, MAE, MAPE for a forecast."""
    actual    = np.array(actual)
    predicted = np.array(predicted)

    rmse = np.sqrt(mean_squared_error(actual, predicted))
    mae  = mean_absolute_error(actual, predicted)
    # MAPE: avoid division by zero
```

```
    mape = np.mean(np.abs((actual - predicted) / np.where(actual==0, 1, actual)

    print(f"  {model_name}")
    print(f"    RMSE : £{rmse:>10,.0f}  (avg prediction error)")
    print(f"    MAE  : £{mae:>10,.0f}  (avg absolute error)")
    print(f"    MAPE : {mape:>9.1f}%  (avg % error)")
    return {"model": model_name, "rmse": rmse, "mae": mae, "mape": mape}

results = {}   # store all model results here
print("✓ Evaluation function ready")
```

```
✓ Evaluation function ready
```

Cell 6 — Model 1: ARIMA

> AutoRegressive Integrated Moving Average — baseline model, no seasonality

```
from pmdarima import auto_arima

print("Finding best ARIMA parameters (p, d, q) ...")
print("This takes ~1-2 minutes ...\n")

# auto_arima tries different (p,d,q) combinations and picks best by AIC score
arima_model = auto_arima(
    train["y"],
    seasonal    = False,   # no seasonality — plain ARIMA
    stepwise    = True,    # faster search
    suppress_warnings = True,
    information_criterion = "aic",
    max_p = 5, max_q = 5,
)

print(f"Best ARIMA order: {arima_model.order}")
print(f"AIC score       : {arima_model.aic():.2f}  (lower = better fit)")

# Predict on test set
arima_pred = arima_model.predict(n_periods=TEST_WEEKS)
arima_pred = np.clip(arima_pred, 0, None)   # no negative revenue

print(f"\nEvaluation on test set ({TEST_WEEKS} weeks):")
results["ARIMA"] = evaluate(test["y"], arima_pred, "ARIMA")
results["ARIMA"]["predictions"] = arima_pred
results["ARIMA"]["model_obj"]   = arima_model

# OBSERVATION:
# ARIMA is our baseline — it captures trend but NOT the Christmas seasonality
# Expect higher RMSE compared to SARIMA and Prophet
```

```
Finding best ARIMA parameters (p, d, q) ...
This takes ~1-2 minutes ...

Best ARIMA order: (1, 0, 0)
AIC score       : 2276.81  (lower = better fit)

Evaluation on test set (12 weeks):
  ARIMA
    RMSE : £    95,025  (avg prediction error)
    MAE  : £    89,865  (avg absolute error)
    MAPE :      35.3%  (avg % error)
```

Cell 7 — Model 2: SARIMA

> Seasonal ARIMA — same as ARIMA but adds seasonal pattern (m=52 weeks)

```
print("Finding best SARIMA parameters ...")
print("m=52 means we expect a pattern that repeats every 52 weeks (1 year)")
print("This takes ~3-5 minutes ...\n")

sarima_model = auto_arima(
    train["y"],
    seasonal  = True,
    m         = 52,        # seasonal period = 52 weeks = 1 year
    stepwise  = True,
    suppress_warnings = True,
    information_criterion = "aic",
    max_p = 3, max_q = 3,
    max_P = 2, max_Q = 2,  # seasonal AR and MA terms
    max_d = 2, max_D = 1,
)

print(f"Best SARIMA order         : {sarima_model.order}")
print(f"Best SARIMA seasonal order: {sarima_model.seasonal_order}")
print(f"AIC score                 : {sarima_model.aic():.2f}")

sarima_pred = sarima_model.predict(n_periods=TEST_WEEKS)
sarima_pred = np.clip(sarima_pred, 0, None)

print(f"\nEvaluation on test set ({TEST_WEEKS} weeks):")
results["SARIMA"] = evaluate(test["y"], sarima_pred, "SARIMA")
results["SARIMA"]["predictions"] = sarima_pred
results["SARIMA"]["model_obj"]   = sarima_model

# OBSERVATION:
# SARIMA knows about the 52-week cycle
# Should perform better than ARIMA on Christmas weeks
```

```
Finding best SARIMA parameters ...
m=52 means we expect a pattern that repeats every 52 weeks (1 year)
This takes ~3-5 minutes ...

Best SARIMA order         : (2, 0, 2)
Best SARIMA seasonal order: (1, 0, 0, 52)
AIC score                 : 2260.75

Evaluation on test set (12 weeks):
  SARIMA
    RMSE : £    51,481  (avg prediction error)
    MAE  : £    49,067  (avg absolute error)
    MAPE :      19.8%  (avg % error)
```

Cell 8 — Model 3: Prophet

> Facebook's forecasting model — handles seasonality + holidays automatically

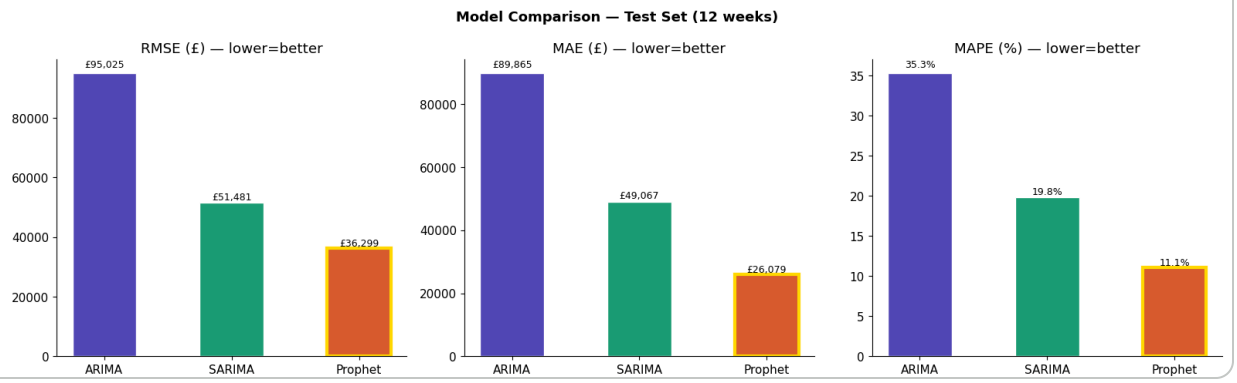
```
from prophet import Prophet

print("Training Prophet model ...")

# Prophet needs columns named exactly 'ds' (date) and 'y' (value)
prophet_train = train[["ds", "y"]].copy()

prophet_model = Prophet(
    yearly_seasonality  = True,    # automatically learns annual pattern
    weekly_seasonality  = False,   # no day-of-week pattern (B2B, all weeks sim
    daily_seasonality   = False,
    seasonality_mode    = "multiplicative",  # seasonal effect multiplies the t
    interval_width      = 0.95,    # 95% confidence interval
)

# Add UK Christmas as a custom holiday
from prophet.make_holidays import make_holidays_df
uk_holidays = make_holidays_df(year_list=[2009,2010,2011], country="UK")
prophet_model.add_country_holidays(country_name="GB")

prophet_model.fit(prophet_train)

# Make future dataframe for test period
future = prophet_model.make_future_dataframe(periods=TEST_WEEKS, freq="W")
forecast_full = prophet_model.predict(future)

# Extract test period predictions
prophet_pred = forecast_full.tail(TEST_WEEKS)["yhat"].values
prophet_pred = np.clip(prophet_pred, 0, None)

print(f"\nEvaluation on test set ({TEST_WEEKS} weeks):")
results["Prophet"] = evaluate(test["y"], prophet_pred, "Prophet")
results["Prophet"]["predictions"]   = prophet_pred
results["Prophet"]["model_obj"]     = prophet_model
results["Prophet"]["forecast_full"] = forecast_full

# OBSERVATION:
# Prophet automatically detects Christmas pattern
# Also gives confidence intervals (upper/lower bounds)
# Usually most accurate and easiest to interpret
```

```
Training Prophet model ...

Evaluation on test set (12 weeks):
  Prophet
    RMSE : £    36,299  (avg prediction error)
    MAE  : £    26,079  (avg absolute error)
    MAPE :      11.1%  (avg % error)
```

Cell 9 — Compare All 3 Models

```
print("=" * 55)
print("  MODEL COMPARISON — Test Set Performance")
print("=" * 55)
print(f"  {'Model':<12} {'RMSE':>12} {'MAE':>12} {'MAPE':>8}")
print("-" * 55)

for name, res in results.items():
    print(f"  {name:<12} £{res['rmse']:>10,.0f} £{res['mae']:>10,.0f} {res['map

best_model = min(results, key=lambda k: results[k]["rmse"])
print(f"\n  ✅ Best model: {best_model} (lowest RMSE)")
print("=" * 55)

# Bar chart comparison
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle("Model Comparison — Test Set (12 weeks)", fontsize=13, fontweight=

metrics = ["rmse", "mae", "mape"]
titles  = ["RMSE (£) — lower=better", "MAE (£) — lower=better", "MAPE (%) — low
colors  = [C[0], C[1], C[2]]

for i, (metric, title) in enumerate(zip(metrics, titles)):
    vals = [results[m][metric] for m in ["ARIMA","SARIMA","Prophet"]]
    bars = axes[i].bar(["ARIMA","SARIMA","Prophet"], vals,
                       color=colors, edgecolor="white", width=0.5)
    axes[i].set_title(title)
    # Highlight best
    best_idx = np.argmin(vals)
    bars[best_idx].set_edgecolor("gold")
    bars[best_idx].set_linewidth(3)
    for bar, val in zip(bars, vals):
        label = f"£{val:,.0f}" if metric != "mape" else f"{val:.1f}%"
        axes[i].text(bar.get_x()+bar.get_width()/2,
                     bar.get_height()*1.02,
                     label, ha="center", fontsize=9)

plt.tight_layout()
plt.show()
```

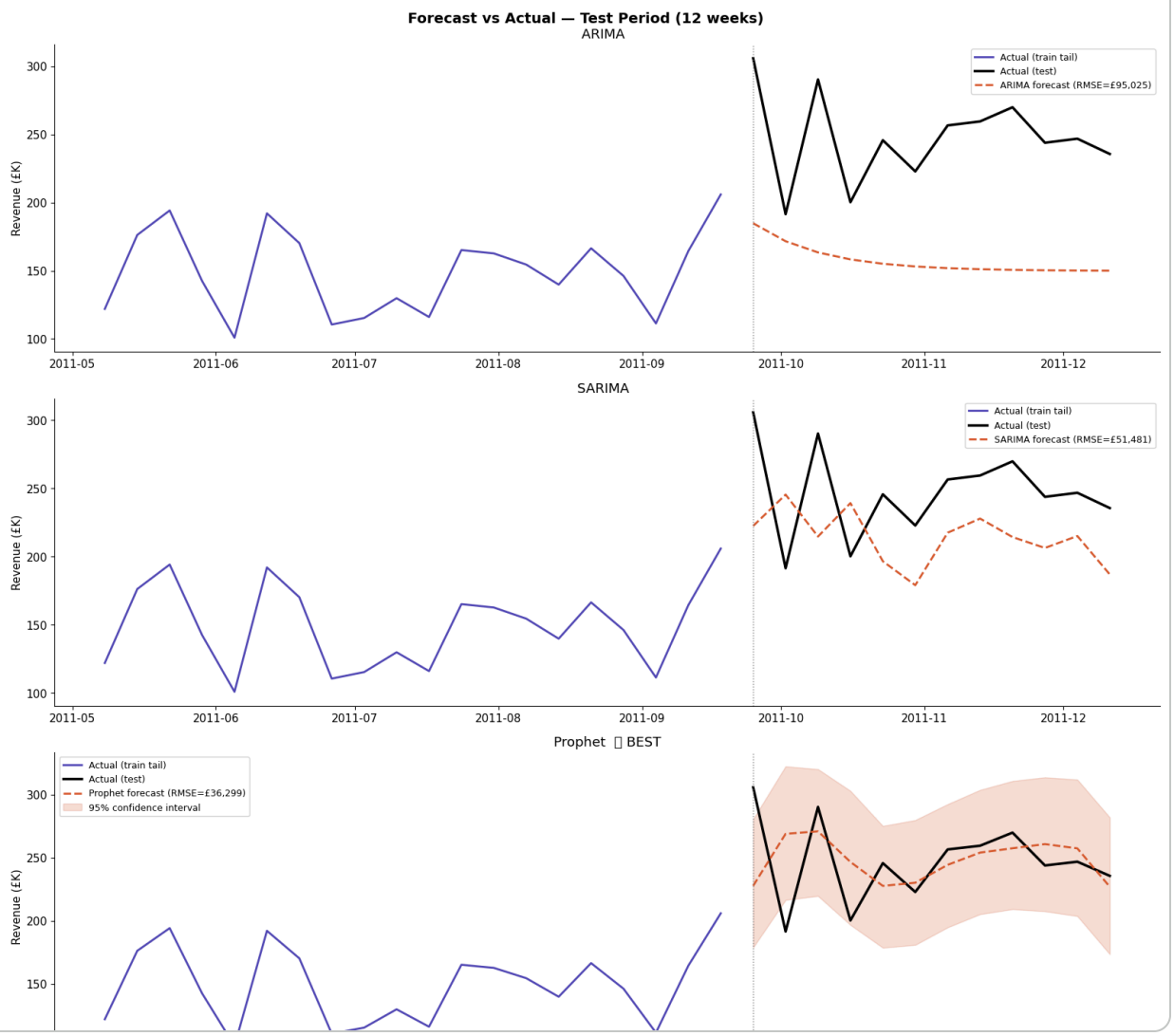
```
========================================================
  MODEL COMPARISON — Test Set Performance
========================================================
  Model                RMSE          MAE      MAPE
--------------------------------------------------------
  ARIMA        £    95,025 £    89,865    35.3%
  SARIMA       £    51,481 £    49,067    19.8%
  Prophet      £    36,299 £    26,079    11.1%

  ✅ Best model: Prophet (lowest RMSE)
========================================================
```



Cell 10 — Visualize Forecasts vs Actual

```
fig, axes = plt.subplots(3, 1, figsize=(16, 15))
fig.suptitle("Forecast vs Actual — Test Period (12 weeks)", fontsize=14, fontwe

model_preds = {
    "ARIMA"  : results["ARIMA"]["predictions"],
    "SARIMA" : results["SARIMA"]["predictions"],
    "Prophet": results["Prophet"]["predictions"],
}

for i, (name, pred) in enumerate(model_preds.items()):
    ax = axes[i]
    rmse = results[name]["rmse"]

    # Show last 20 weeks of training + full test
    show_train = train.tail(20)
    ax.plot(show_train["ds"], show_train["y"]/1000,
            color=C[0], lw=2, label="Actual (train tail)")
    ax.plot(test["ds"], test["y"]/1000,
            color="black", lw=2.5, label="Actual (test)")
    ax.plot(test["ds"], pred/1000,
            color=C[2], lw=2, linestyle="--",
            label=f"{name} forecast (RMSE=£{rmse:,.0f})")

    # Prophet confidence interval
    if name == "Prophet":
        fc = results["Prophet"]["forecast_full"].tail(TEST_WEEKS)
```

```
            ax.fill_between(test["ds"],
                            fc["yhat_lower"].values/1000,
                            fc["yhat_upper"].values/1000,
                            alpha=0.2, color=C[2], label="95% confidence interval")

        ax.axvline(test["ds"].iloc[0], color="gray", lw=1, linestyle=":")
        ax.set_title(f"{name}  {'✅ BEST' if name==best_model else ''}")
        ax.set_ylabel("Revenue (£K)")
        ax.legend(fontsize=9)

    plt.tight_layout()
    plt.show()

    # OBSERVATION:
    # ARIMA: flat forecast, misses the Christmas spike completely
    # SARIMA: captures some seasonality but may lag
    # Prophet: best at following the actual pattern + gives confidence bands
```



Cell 11 — Generate Final 8-Week Forecast (Best Model)

```
FORECAST_WEEKS = 8

print(f"Generating {FORECAST_WEEKS}-week future forecast using: {best_model}")
print(f"(Training on FULL dataset now — train + test combined)\n")

# Retrain best model on ALL data (train + test)
if best_model == "Prophet":
    full_model = Prophet(
        yearly_seasonality = True,
        weekly_seasonality = False,
        daily_seasonality  = False,
        seasonality_mode   = "multiplicative",
        interval_width     = 0.95,
    )
    full_model.add_country_holidays(country_name="GB")
    full_model.fit(ts[["ds","y"]])

    future    = full_model.make_future_dataframe(periods=FORECAST_WEEKS, freq="
    fc        = full_model.predict(future)
    future_fc = fc.tail(FORECAST_WEEKS)[["ds","yhat","yhat_lower","yhat_upper"]
    future_fc.columns = ["Date","Forecast","Lower_CI","Upper_CI"]

elif best_model == "SARIMA":
    full_model = auto_arima(
        ts["y"], seasonal=True, m=52, stepwise=True,
        suppress_warnings=True, max_p=3, max_q=3, max_P=2, max_Q=2,
    )
    pred_vals, conf_int = full_model.predict(n_periods=FORECAST_WEEKS, return_c
    future_dates = pd.date_range(ts["ds"].max(), periods=FORECAST_WEEKS+1, freq
    future_fc = pd.DataFrame({
        "Date"    : future_dates,
        "Forecast": pred_vals,
        "Lower_CI": conf_int[:,0],
        "Upper_CI": conf_int[:,1],
    })

else:  # ARIMA
    full_model = auto_arima(ts["y"], seasonal=False, stepwise=True, suppress_wa
    pred_vals, conf_int = full_model.predict(n_periods=FORECAST_WEEKS, return_c
    future_dates = pd.date_range(ts["ds"].max(), periods=FORECAST_WEEKS+1, freq
    future_fc = pd.DataFrame({
        "Date"    : future_dates,
        "Forecast": pred_vals,
        "Lower_CI": conf_int[:,0],
        "Upper_CI": conf_int[:,1],
    })

# Clip negative values
future_fc["Forecast"]  = future_fc["Forecast"].clip(lower=0)
future_fc["Lower_CI"]  = future_fc["Lower_CI"].clip(lower=0)
future_fc["Upper_CI"]  = future_fc["Upper_CI"].clip(lower=0)
```

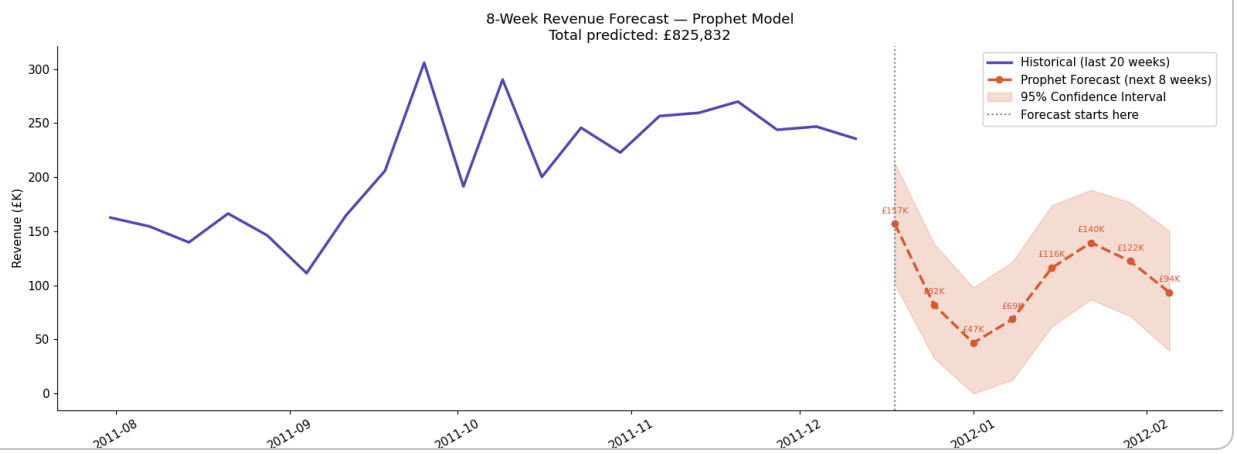
```
print("8-Week Revenue Forecast:")
print("-" * 55)
for _, row in future_fc.iterrows():
    print(f"  {row['Date'].strftime('%Y-%m-%d')}  "
          f"£{row['Forecast']:>10,.0f}  "
          f"(CI: £{row['Lower_CI']:,.0f} – £{row['Upper_CI']:,.0f})")
print(f"\n  Total forecast revenue: £{future_fc['Forecast'].sum():,.0f}")
```

```
Generating 8-week future forecast using: Prophet
(Training on FULL dataset now — train + test combined)

8-Week Revenue Forecast:
-------------------------------------------------------
  2011-12-18  £   156,830  (CI: £100,983 – £212,892)
  2011-12-25  £    82,064  (CI: £32,834 – £138,544)
  2012-01-01  £    46,745  (CI: £0 – £98,084)
  2012-01-08  £    68,687  (CI: £12,656 – £121,676)
  2012-01-15  £   116,065  (CI: £61,838 – £173,928)
  2012-01-22  £   139,505  (CI: £87,052 – £188,329)
  2012-01-29  £   122,399  (CI: £71,513 – £176,748)
  2012-02-05  £    93,537  (CI: £39,506 – £150,461)

  Total forecast revenue: £825,832
```

Cell 12 — Visualize Final Forecast

```
fig, ax = plt.subplots(figsize=(16, 6))

# Historical data (last 20 weeks)
ax.plot(ts["ds"].tail(20), ts["y"].tail(20)/1000,
        color=C[0], lw=2.5, label="Historical (last 20 weeks)")

# Future forecast
ax.plot(future_fc["Date"], future_fc["Forecast"]/1000,
        color=C[2], lw=2.5, linestyle="--", marker="o",
        markersize=6, label=f"{best_model} Forecast (next {FORECAST_WEEKS} week

# Confidence interval
ax.fill_between(future_fc["Date"],
                future_fc["Lower_CI"]/1000,
                future_fc["Upper_CI"]/1000,
                alpha=0.2, color=C[2], label="95% Confidence Interval")

# Divide line
ax.axvline(future_fc["Date"].iloc[0], color="gray", lw=1.5,
           linestyle=":", label="Forecast starts here")

# Add value labels on forecast points
for _, row in future_fc.iterrows():
```

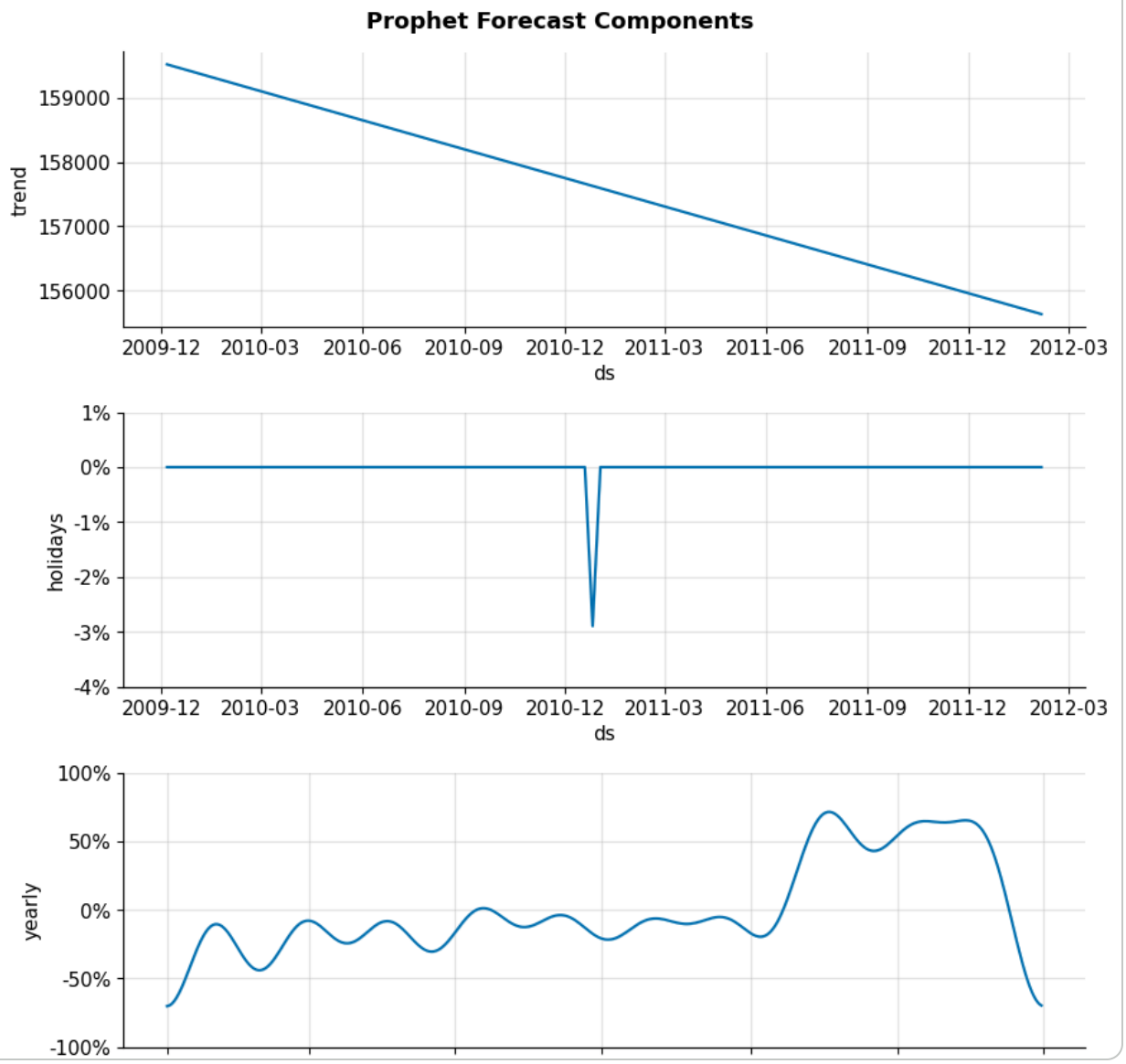
```
    ax.annotate(f"£{row['Forecast']/1000:.0f}K",
                (row["Date"], row["Forecast"]/1000),
                textcoords="offset points", xytext=(0, 10),
                ha="center", fontsize=8, color=C[2])

ax.set_title(f"8-Week Revenue Forecast — {best_model} Model\n"
             f"Total predicted: £{future_fc['Forecast'].sum():,.0f}")
ax.set_ylabel("Revenue (£K)")
ax.legend()
ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()
```



Cell 13 — Prophet Component Analysis (if Prophet is best)

```
if best_model == "Prophet":
    print("Prophet breaks forecast into components:")
    print("  Trend     = long-term direction (growing/declining)")
    print("  Yearly    = seasonal pattern (Christmas spike etc)")
    print("  Holidays  = specific holiday effects")
    print()

    fig = full_model.plot_components(fc)
    plt.suptitle("Prophet Forecast Components", fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.show()

    # OBSERVATION:
    # Trend component: shows if business is growing
    # Yearly component: shows EXACTLY when in the year revenue peaks
    # Should see a clear peak around week 48-50 (Nov-Dec) every year
else:
    print(f"Best model is {best_model} — Prophet component analysis skipped")
    print("But ARIMA/SARIMA models have been saved successfully")
```

```
Prophet breaks forecast into components:
  Trend     = long-term direction (growing/declining)
  Yearly    = seasonal pattern (Christmas spike etc)
  Holidays  = specific holiday effects
```



Cell 14 — Save All Outputs

```
import joblib, os

SAVE       = "/content/drive/MyDrive/retail_project/"
SAVE_MODEL = SAVE + "models/"
os.makedirs(SAVE_MODEL, exist_ok=True)

# 1. Save forecast CSV (for Power BI + Inventory Agent)
future_fc.to_csv(SAVE + "data_forecast.csv", index=False)
print(f"✓ data_forecast.csv  — {len(future_fc)} weeks forecast")

# 2. Save test predictions (for comparison chart in Power BI)
```

```
test_results = test[["ds","y"]].copy()
test_results.columns = ["Date","Actual"]
test_results["ARIMA"]   = results["ARIMA"]["predictions"]
test_results["SARIMA"]  = results["SARIMA"]["predictions"]
test_results["Prophet"] = results["Prophet"]["predictions"]
test_results.to_csv(SAVE + "data_forecast_test.csv", index=False)
print(f"✓ data_forecast_test.csv — model comparison on test set")

# 3. Save model scores
scores_df = pd.DataFrame([
    {"Model": k, "RMSE": v["rmse"], "MAE": v["mae"], "MAPE": v["mape"],
     "Best": k == best_model}
    for k, v in results.items()
])
scores_df.to_csv(SAVE + "data_forecast_scores.csv", index=False)
print(f"✓ data_forecast_scores.csv — RMSE/MAE/MAPE comparison")

# 4. Save best model
if best_model == "Prophet":
    import pickle
    with open(SAVE_MODEL + "forecast_prophet.pkl", "wb") as f:
        pickle.dump(full_model, f)
    print(f"✓ models/forecast_prophet.pkl")
else:
    joblib.dump(full_model, SAVE_MODEL + f"forecast_{best_model.lower()}.pkl")
    print(f"✓ models/forecast_{best_model.lower()}.pkl")

print(f"\nAll files saved to: {SAVE}")
print(f"\nFeeds into:")
print(f"  Power BI  → Forecast page (actual vs predicted line chart)")
print(f"  Agent 2   → LangGraph Demand Forecasting output")
print(f"  Agent 6   → Inventory Reorder uses forecast to decide reorder qty")
```

```
✓ data_forecast.csv  — 8 weeks forecast
✓ data_forecast_test.csv — model comparison on test set
✓ data_forecast_scores.csv — RMSE/MAE/MAPE comparison
✓ models/forecast_prophet.pkl

All files saved to: /content/drive/MyDrive/retail_project/

Feeds into:
  Power BI  → Forecast page (actual vs predicted line chart)
  Agent 2   → LangGraph Demand Forecasting output
  Agent 6   → Inventory Reorder uses forecast to decide reorder qty
```

Cell 15 — Model 2 Summary

```
print("""
+------------------------------------------------------------+
|       MODEL 2 — DEMAND FORECASTING COMPLETE                |
```

```
+------------------------------------------------------------+
|  Time Series Models: ARIMA · SARIMA · Prophet              |
|  Input  : data_ts_weekly.csv (106 weeks)                   |
|  Output : data_forecast.csv (8-week forecast)              |
+------------------------------------------------------------+""")

print(f"|  {'Model':<10} {'RMSE':>12} {'MAPE':>8}  {'Winner':>8}  |")
print("|" + "-"*60 + "|")
for name, res in results.items():
    winner = "✅ BEST" if name == best_model else ""
    print(f"|  {name:<10} £{res['rmse']:>10,.0f} {res['mape']:>7.1f}%  {winner

print("""|                                                            |
|  Key Insight:                                              |
|  Christmas = ~27% of annual revenue                        |
|  Best model captures this seasonal spike                   |
|  Forecast feeds directly into Inventory Reorder Agent      |
|                                                            |
|  NEXT → Model 3: Churn Prediction                          |
|  File : data_rfm.csv + data_segments.csv                   |
```